RACEWISE

Autonomous Evidence-Driven Race Engineering

Deep Research • Product Requirements • System Architecture

17 September 2026

RACEWISE — Deep Research & Product Direction Brief

Date: 17 September 2026

Working title: RACEWISE — Autonomous Evidence-Driven Race Engineering

Purpose: Establish a genuinely new project that is not a reimplementation of Live F1 Intelligence or the F1 Lap-Time Simulator, and exploit the limited Union Alpha free window as a controlled agentic-AI engineering experiment.

1. Executive conclusion

RACEWISE should not be another telemetry dashboard, race narrator, strategy recommender, or simulator.

Its core problem is:

> Given an engineering question about a race, autonomously construct and test competing explanations using deterministic data tools and counterfactual experiments, then produce a traceable evidence-backed conclusion.

The differentiator is the investigation loop, not the LLM itself.

The system should behave more like a scientific investigator / race-engineering analyst:

1. Frame the question.
2. Generate explicit hypotheses.
3. Select the minimum useful evidence.
4. Call deterministic analysis tools.
5. Compare evidence against hypotheses.
6. Search for contradictory evidence.
7. Run counterfactual experiments where appropriate.
8. Update confidence.
9. Produce a causal/evidentiary report.
10. Preserve the complete investigation trace.

2. Union Alpha research

OpenRouter currently describes Union Alpha as a stealth multimodal model for research, coding and agentic workflows. It exposes a 262K-token context window, image input, tool calling and structured JSON output. It was released 16 September 2026 and is currently free. The provider is anonymous; OpenRouter says prompts/completions may be retained by the provider, though not used for training. Therefore RACEWISE should not send secrets, credentials or proprietary source material to the model.

Union Alpha's benchmark evidence is still immature. Its public benchmark page references Terminal-Bench v4.0 and warns that the model is stealth, the provider is anonymous, the model may change, and current pricing is anticipated rather than established. Independent reporting likewise notes the absence of a complete, reproducible suite of standard frontier benchmarks and mixed early reports about speed and long-running agent reliability.

Decision: Use Union Alpha as an experimental agent runtime, not as an unquestioned source of truth. The project must be useful even if the model is replaced.

3. Existing-project collision analysis

Live F1 Intelligence

Primary question: What is happening?

Typical architecture:

`data ingestion -> normalization -> analytics -> real-time publication -> dashboard`

RACEWISE must not duplicate:

• live telemetry ingestion infrastructure

• timing dashboard surfaces

• WebSocket race feed

• basic sector/tyre visualizations

• generic live narration

RACEWISE may consume these capabilities through adapters.

F1 Lap-Time Simulator

Primary question: What happens under specified engineering parameters?

Typical architecture:

`parameters -> vehicle/tyre/track model -> simulation -> result`

RACEWISE must not duplicate:

• vehicle physics model

• full lap-time simulation UI

• setup optimizer as the primary product

RACEWISE may call a simulator as a counterfactual experiment tool.

RACEWISE

Primary question:

Why did this happen, what competing explanations fit the evidence, and what intervention would change the outcome?

Architecture:

`question -> hypotheses -> evidence plan -> tool calls -> falsification -> counterfactuals -> synthesis -> provenance`

4. Competitive / research landscape

The research scan shows that several adjacent ideas already exist:

• F1 StratLab targets AI strategy recommendations and has moved toward live-race inference.

• OpenF1ow provides transparent post-race analytics and LLM-generated race verdicts.

• Academic work has explored explainable tyre-energy prediction and counterfactual explanations.

• Recent research has explored automated hypothesis validation through sequential falsification.

• Recent Nature work such as Robin and Co-Scientist demonstrates a broader pattern: AI systems can iteratively generate hypotheses, use specialized tools/agents, analyze results, critique hypotheses, and refine conclusions.

This means a generic "AI race strategist" is not sufficiently differentiated.

The defensible novelty for a one-week portfolio build is to bring the hypothesis → evidence → falsification → counterfactual → provenance pattern into an F1 engineering environment, with deterministic analysis tools separating computation from language-model interpretation.

5. Data feasibility

OpenF1 provides historical F1 data from 2023 onward without authentication, including car data, laps, intervals, location, pit stops, position, race control, sessions, stints, weather and limited team radio. Real-time data requires a paid subscription. OpenF1 explicitly describes itself as independent and intended for educational, research and personal use.

FastF1 exposes session, timing, telemetry, results, circuit information and related APIs. It is a suitable historical-analysis adapter.

Important limitation: team-radio availability has decreased significantly in 2026 and is not guaranteed. RACEWISE must treat radio as optional evidence, never a required dependency.

6. Research-derived product principles

Principle A — AI is the investigator, not the calculator

Numerical values must come from deterministic tools.

Bad:

`LLM: "Tyre degradation is 0.37 seconds/lap."`

Good:

`TyreAnalyzer -> 0.37 s/lap -> agent interprets significance.`

Principle B — Every conclusion must be traceable

Every important claim should reference:

• data source

• query/tool

• analysis method/version

• time/lap window

• assumptions

• confidence

• contradictory evidence, if found

Principle C — Falsification is a first-class action

The agent must be able to ask:

`What evidence would make my current hypothesis wrong?`

It should actively search for that evidence before finalizing a strong conclusion.

Principle D — Counterfactuals must be simulations, not prose

"What if we pitted five laps earlier?" must call a deterministic scenario engine.

Principle E — Preserve uncertainty

The system must distinguish:

• observed fact

• calculated metric

• inferred explanation

• counterfactual estimate

• model assumption

Principle F — Model-agnostic core

The investigation engine must expose a provider interface so Union Alpha can be replaced by another model without changing domain logic.

7. Proposed core innovation

Evidence-backed Investigation Graph

Each investigation becomes a graph:

`Question -> Hypothesis -> Evidence Request -> Tool Execution -> Observation -> Test -> Counter-evidence -> Updated Hypothesis -> Counterfactual -> Conclusion`

The UI can show this graph as a replayable investigation trace.

Example

Question:

"Why did Driver X lose 0.8s/lap in the second stint?"

Hypotheses:

• tyre degradation

• traffic

• fuel/load effect

• weather

• driver management

• car performance

• strategy

The agent gathers evidence, rejects weak explanations, tests alternatives, and only then synthesizes a conclusion.

8. One-week feasibility

Scope is intentionally constrained to a vertical slice:

• 1–3 historical races

• 1 primary OpenF1 adapter

• 8–12 deterministic analysis tools

• 1 simplified counterfactual strategy engine

• 1 Union Alpha agent

• 1 investigation graph

• 1 evidence/provenance layer

• polished React UI

• evaluation suite of 15–20 investigation questions

Do not build a new live timing system, full F1 simulator, multi-season ML platform or production-grade authentication system during the one-week sprint.

9. Portfolio value

The project demonstrates:

• agentic architecture

• tool calling

• long-context reasoning

• multimodal analysis

• deterministic analytics

• causal reasoning discipline

• counterfactual simulation

• provenance

• evaluation

• F1 domain expertise

• full-stack engineering

The strongest portfolio claim is not "I used Union Alpha."

It is:

> "I built an evidence-driven autonomous investigation system in which an AI agent forms and falsifies engineering hypotheses using deterministic motorsport analysis tools and counterfactual simulation."

10. Sources

• OpenRouter — Union Alpha: https://openrouter.ai/stealth/union-alpha/

• Union Alpha benchmark page: https://unionalpha.io/benchmarks.html

• Union Alpha usage / OpenCode: https://unionalpha.io/how-to-use-union-alpha.html

• OpenF1 documentation: https://openf1.org/docs/

• OpenF1: https://openf1.org/

• FastF1 API reference: https://github.com/theOehrly/Fast-F1

• Automated Hypothesis Validation / POPPER: https://proceedings.mlr.press/v267n.html

• Nature — Robin: https://www.nature.com/articles/s41586-026-10652-y

• Nature — Co-Scientist: https://www.nature.com/articles/s41586-026-10644-y

• Explainable F1 tyre-energy prediction: https://arxiv.org/abs/2501.04067

• F1 tyre degradation state-space modeling: https://journals.sagepub.com/doi/10.1177/22150218261446170

• Pitwall race-strategy grounding research: https://arxiv.org/abs/2607.06495

• F1 StratLab: https://f1stratlab.com/

RACEWISE — Product Requirements Document

Version: 0.1.0

Date: 17 September 2026

Status: Development-ready MVP specification

1. Product definition

RACEWISE (Autonomous Evidence-Driven Race Engineering) is an AI-assisted investigation platform for Formula 1 engineering questions.

RACEWISE does not primarily tell the user what happened. It investigates why an observed outcome occurred and tests alternative explanations.

Product promise

> Ask a race-engineering question. RACEWISE constructs hypotheses, gathers evidence through deterministic tools, attempts to falsify its own explanations, runs counterfactual scenarios, and produces a traceable engineering report.

2. Problem

F1 analytics systems commonly expose metrics and visualizations. Strategy systems can recommend actions. Simulators can estimate outcomes.

The missing portfolio-scale interaction is an integrated loop that connects:

• an open-ended engineering question

• explicit competing hypotheses

• evidence retrieval

• deterministic analysis

• falsification

• counterfactual experimentation

• uncertainty

• provenance

RACEWISE fills that gap as an experimental research/engineering system.

3. Target users

Primary

• Motorsport engineering students

• Performance-engineering enthusiasts

• Data scientists

• AI/agent researchers

• F1 analysts

Secondary

• Recruiters and technical interviewers evaluating engineering/AI portfolios

• Researchers exploring agentic decision-support systems

4. MVP user journeys

Journey A — Explain an anomaly

Input:

"Why did Driver 16 lose pace from laps 28–36?"

System:

1. creates hypotheses
2. selects evidence
3. calls analysis tools
4. evaluates evidence
5. runs contradiction checks
6. produces conclusion and uncertainty

Journey B — Challenge a conclusion

User clicks `Challenge conclusion`.

System:

1. extracts current claims
2. generates falsification tests
3. searches for contradictory evidence
4. revises confidence
5. displays what changed

Journey C — Counterfactual strategy

User asks:

"What if the pit stop happened 5 laps earlier?"

System:

1. freezes observed state
2. calls strategy simulator
3. compares baseline vs counterfactual
4. reports estimated delta and assumptions

Journey D — Compare drivers

Question:

"Was Driver A genuinely slower, or did traffic explain the gap?"

System compares matched/clean laps and separates observed pace from contextual effects.

5. Functional requirements

FR-01 Investigation creation

The system shall accept a natural-language engineering question plus race/session/driver context.

FR-02 Hypothesis generation

The agent shall generate multiple plausible hypotheses with measurable predictions.

Each hypothesis must contain:

• id

• statement

• predicted observable

• required evidence

• potential falsifier

FR-03 Evidence planning

The agent shall select tools based on information value rather than blindly calling every endpoint.

FR-04 Deterministic tools

All quantitative calculations shall be performed by versioned Python services/functions.

Initial tools:

• session metadata

• lap extraction

• clean-lap filtering

• sector comparison

• stint comparison

• tyre-age degradation

• traffic exposure

• pit-loss analysis

• weather correlation

• race-control event correlation

• driver comparison

• strategy counterfactual

FR-05 Tool execution trace

Every tool call shall record:

• tool name/version

• parameters

• timestamp

• result hash

• execution status

FR-06 Hypothesis evaluation

Each hypothesis shall receive:

• supporting evidence

• contradictory evidence

• unresolved evidence

• confidence

• status: supported / weakened / unresolved / rejected

FR-07 Falsification

The agent shall explicitly attempt to find evidence that could invalidate its leading hypothesis.

FR-08 Counterfactual simulation

The system shall support at least:

• earlier pit

• later pit

• compound change

The simulator must expose assumptions and uncertainty.

FR-09 Provenance

Every final claim shall link to evidence nodes.

FR-10 Final report

Reports shall contain:

1. question

2. observed facts

3. hypotheses considered

4. evidence

5. contradictions

6. counterfactuals

7. conclusion

8. confidence

9. limitations

10. reproducibility metadata

FR-11 Investigation replay

A completed investigation shall be replayable from its event log.

FR-12 Model abstraction

Union Alpha shall be one provider implementation, not a hard-coded architectural dependency.

FR-13 Image evidence

The agent may inspect generated telemetry/strategy plots and use them as secondary evidence.

6. Non-functional requirements

Reliability

• deterministic tool outputs

• schema validation

• retries

• idempotent tool execution

Explainability

No high-level conclusion without evidence references.

Safety / trust

The UI must visually distinguish:

`FACT`, `CALCULATION`, `INFERENCE`, `COUNTERFACTUAL`, `ASSUMPTION`.

Performance

MVP target:

• initial question acknowledgement < 2 s

• tool calls streamed progressively

• investigation visible incrementally

• no blocking requirement for model completion

Reproducibility

Each investigation stores:

• data source version

• model identifier

• prompt version

• tool versions

• configuration

• random seeds where applicable

7. UI requirements

Screen 1 — Investigation Workspace

• race/session selector

• driver selector

• question input

• investigation status

• live agent trace

Screen 2 — Hypothesis Board

Cards for each hypothesis:

• claim

• evidence score

• confidence

• supporting evidence

• contradictory evidence

• status

Screen 3 — Evidence Graph

Interactive graph:

`Question -> Hypothesis -> Evidence -> Test -> Result -> Conclusion`

Screen 4 — Counterfactual Lab

Baseline and scenario comparison:

• pit lap

• compound

• predicted pace

• predicted race delta

• assumptions

Screen 5 — Engineering Report

A clean report suitable for export/share.

8. Evaluation requirements

Build a fixed benchmark set of 15–20 questions.

Categories:

• tyre degradation

• traffic

• strategy

• weather

• driver comparison

• pit timing

• anomalous pace

• safety-car effects

• counterfactuals

Score:

• factual accuracy

• numerical accuracy

• tool-selection efficiency

• evidence coverage

• contradiction detection

• hallucination rate

• conclusion calibration

• reproducibility

• latency

Do not claim Union Alpha is superior to other models unless the experiment actually demonstrates it.

9. Explicit non-goals

MVP will NOT include:

• licensed F1 live data

• a full race broadcast product

• a replacement for Live F1 Intelligence

• a replacement for the F1 simulator

• autonomous real-world race control

• production financial/payment systems

• full multi-team race strategy optimization

• unrestricted web browsing by the agent

• unverified LLM-generated numerical calculations

10. Definition of done

MVP is complete when:

• one historical race can be investigated end-to-end

• at least 8 deterministic tools work

• agent can generate and evaluate ≥3 hypotheses

• agent performs at least one explicit falsification step

• one counterfactual strategy scenario executes deterministically

• every final conclusion has provenance

• investigation can be replayed

• 15–20 benchmark questions are scored

• polished web UI is deployable

• README contains architecture, limitations and evaluation results

11. Seven-day plan

Day 1

Repo, schemas, OpenF1 adapter, storage, model provider abstraction.

Day 2

Deterministic analysis tools and evidence model.

Day 3

Investigation orchestrator, hypothesis schema, tool calling, Union Alpha integration.

Day 4

Falsification loop + counterfactual strategy engine.

Day 5

React investigation workspace + evidence graph + trace.

Day 6

Evaluation harness + report export + robustness fixes.

Day 7

UI polish, deployment, documentation, benchmark, demo recording.

12. Portfolio deliverables

• deployed web application

• GitHub repository

• architecture diagram

• PRD

• benchmark report

• investigation examples

• 2–3 minute demo

• technical write-up

• reproducibility manifest

13. Success metrics

The MVP succeeds if:

• ≥90% of deterministic numerical outputs match reference calculations

• ≥90% of final factual claims have evidence links

• falsification is attempted on ≥95% of eligible investigations

• ≥80% of benchmark investigations reach a defensible conclusion

• replay produces equivalent deterministic tool outputs

• model can be swapped without domain-code changes

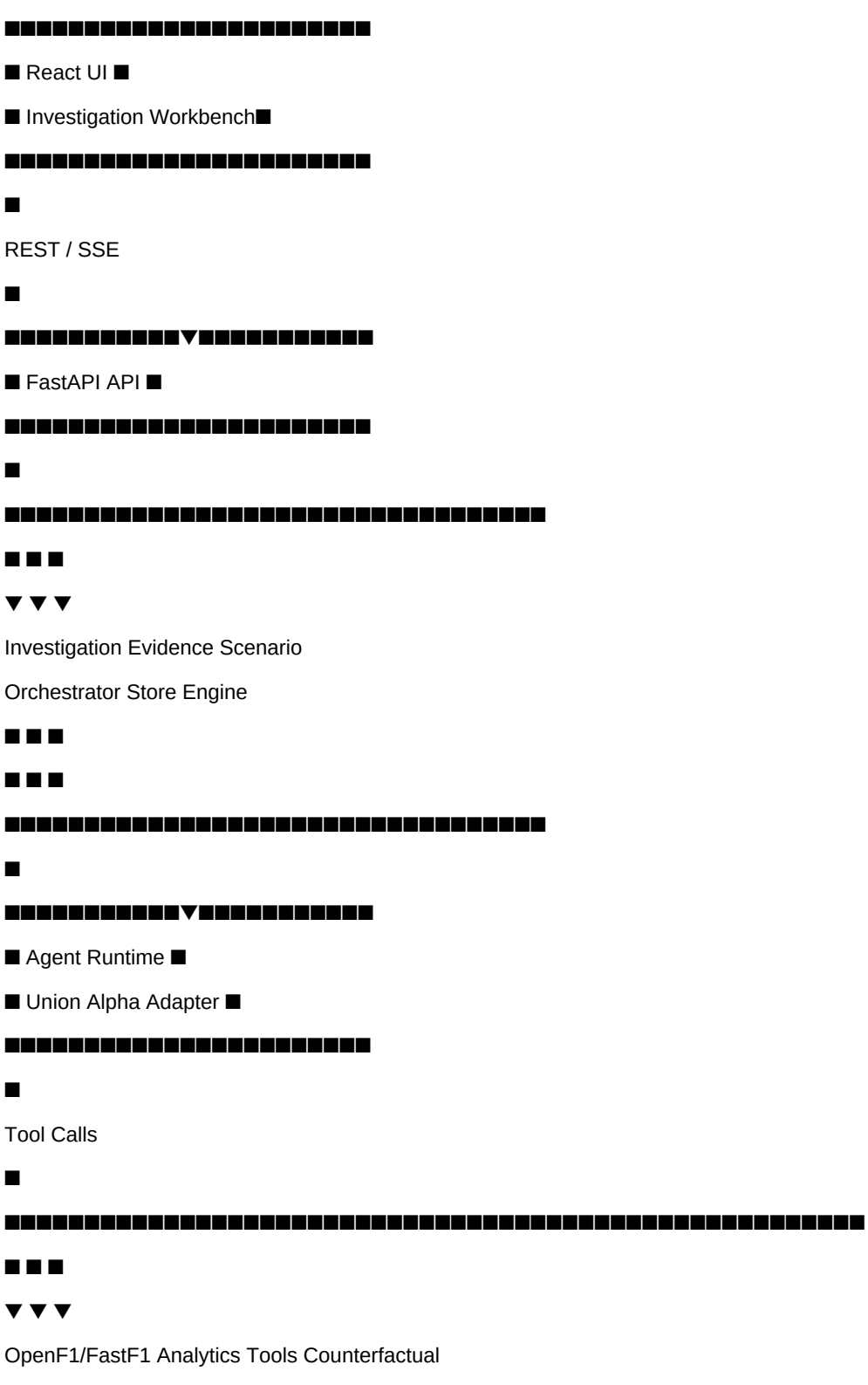
RACEWISE — System Architecture

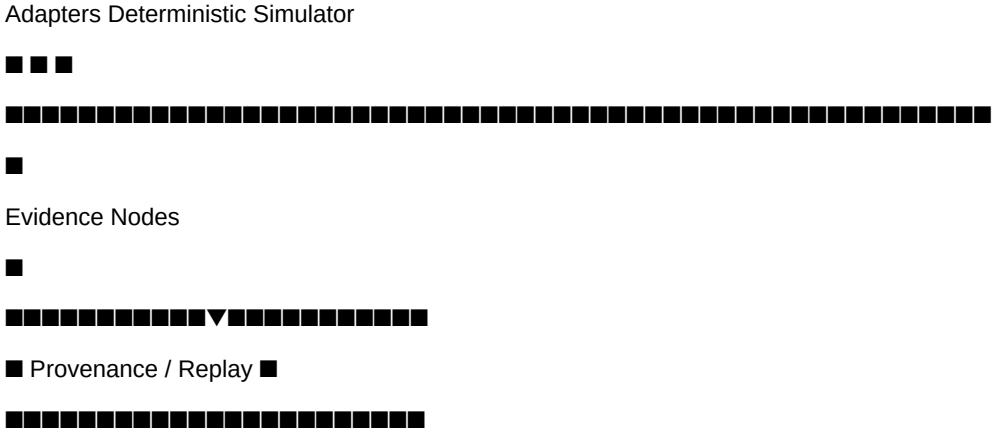
Version: 0.1.0

Date: 17 September 2026

Architecture principle: AI orchestrates investigation; deterministic software owns computation.

1. System overview





2. Architectural separation

Layer 1 — Data

Responsibilities:

• fetch historical session data

• cache raw responses

• normalize into canonical internal schemas

Adapters:

• OpenF1

• FastF1

The application must never expose provider-specific JSON directly to the agent.

3. Canonical domain model

Session

Session(

session_id,

meeting_id,

event_name,

session_type,

date,

circuit

)

Lap

Lap(

session_id,

driver_id,

lap_number,

lap_time,

sector_1,

sector_2,

sector_3,

compound,

tyre_age,

pit_in,

pit_out,

is_valid

)

Observation

Observation(

id,

metric,

value,

unit,

time_window,

source,

method_version

)

Hypothesis

Hypothesis(

id,

statement,

predicted_observables,

falsifiers,

status,

confidence

)

Evidence

Evidence(

id,

type,

observation_ids,

tool_execution_id,

supports,

contradicts,

strength

)

Counterfactual

Counterfactual(

baseline_id,

intervention,

assumptions,

predicted_delta,

uncertainty,

simulator_version

)

Conclusion

Conclusion(

statement,

confidence,

evidence_ids,

counterfactual_ids,

limitations

)

4. Investigation state machine

CREATED

■

▼

FRAMING

■

▼

HYPOTHESIS_GENERATION

■

▼

EVIDENCE_PLANNING

■

▼

EVIDENCE_COLLECTION

■

▼

HYPOTHESIS_TESTING

■

■■■■■■■■■■■■■■■■■

■ ■

▼ ▼

FALSIFICATION MORE_DATA

■ ■

■■■■■■■■■■■■■■■■■

▼

COUNTERFACTUAL_ANALYSIS

■

▼

SYNTHESIS

■

▼

VERIFICATION

■

▼

COMPLETED

Failure states:

• `DATA_UNAVAILABLE`

• `TOOL_FAILURE`

• `MODEL_FAILURE`

• `INSUFFICIENT_EVIDENCE`

5. Agent architecture

The MVP uses one model with role-structured prompts rather than a swarm of independent LLMs.

Agent roles

Supervisor

Frames the question and controls the investigation state.

Hypothesis planner

Produces competing explanations and falsifiers.

Evidence planner

Selects tools.

Investigator

Interprets tool results.

Falsifier

Attempts to disprove the leading explanation.

Synthesizer

Produces the final report.

These can be implemented as prompt phases over one Union Alpha session initially. The architecture must permit splitting them into separate agents later.

6. Tool contract

Every tool must implement:

Tool(

name,

description,

input_schema,

output_schema,

version,

execute()

)

Example:

analyze_tyre_degradation(

session_id,

driver_id,

stint_id,

clean_lap_policy

) -> TyreDegradationResult

The agent receives:

• what the tool does

• accepted parameters

• expected output

• known limitations

The agent does NOT receive arbitrary database access.

7. Core tool catalogue

Data

• `get_session`

• `get_driver`

• `get_laps`

• `get_stints`

• `get_weather`

• `get_race_control`

• `get_pit_stops`

• `get_team_radio`

Analysis

• `filter_clean_laps`

• `compare_sectors`

• `analyze_tyre_degradation`

• `analyze_traffic`

• `analyze_pit_loss`

• `compare_driver_pace`

• `correlate_weather`

• `detect_pace_anomaly`

Counterfactual

• `simulate_pit_timing`

• `simulate_compound_change`

• `compare_strategy_scenarios`

8. Evidence model

The system must distinguish four evidence classes:

FACT

Directly retrieved from a source.

CALCULATION

Computed deterministically from facts.

INFERENCE

Agent interpretation supported by evidence.

COUNTERFACTUAL

Model/simulator output under explicit assumptions.

UI must use distinct labels for these classes.

9. Falsification engine

Pseudo-flow:

```
leading = select_leading_hypothesis(hypotheses)

tests = generate_falsification_tests(
hypothesis=leading,
evidence=current_evidence
)

for test in tests:
result = execute(test)
update_hypothesis(leading, result)

if contradictory_evidence_is_material():
downgrade_confidence()
```

The system should prefer tests that can materially distinguish between competing hypotheses.

10. Counterfactual engine

The simulator is intentionally simpler than the existing F1 Lap-Time Simulator.

Inputs:

- observed race state
- intervention
- tyre assumptions
- pit loss
- traffic assumptions
- remaining laps

Outputs:

- predicted time delta
- position implications where model permits
- uncertainty
- assumptions
- simulation version

It must not pretend to reproduce an actual team's race simulator.

11. Provenance architecture

Claim

■

■■■ Evidence

■ ■■■ Observation

■ ■■■ Tool Execution

■ ■■■ Source Record

■

■■■ Method Version

■■■ Model Version

■■■ Investigation ID

Every final claim should be reconstructible.

Event log

{

"event_id": "...",

"investigation_id": "...",

"type": "tool_execution",

"tool": "analyze_tyre_degradation",

"version": "1.0.0",

"input_hash": "...",

"output_hash": "...",

"timestamp": "..."

}

12. Storage

MVP:

• SQLite for local development

• PostgreSQL for deployment

• object storage optional for generated plots/reports

Tables:

• investigations

• hypotheses

• evidence

• observations

• tool_executions

• counterfactuals

• conclusions

• event_log

• datasets

13. API

POST

`/api/investigations`

Creates an investigation.

GET

`/api/investigations/{id}`

Returns current state.

GET

`/api/investigations/{id}/events`

Returns replayable event stream.

POST

`/api/investigations/{id}/challenge`

Starts falsification.

POST

`/api/investigations/{id}/counterfactual`

Runs a scenario.

GET

`/api/investigations/{id}/report`

Returns final report.

GET

`/api/sessions`

Lists supported sessions.

14. Streaming

Use Server-Sent Events for MVP:

investigation.started

hypothesis.created

tool.started

tool.completed

evidence.added

hypothesis.updated

falsification.started

counterfactual.started

counterfactual.completed

synthesis.started

investigation.completed

WebSockets are unnecessary for the first vertical slice.

15. Model abstraction

class ModelProvider(Protocol):

def respond(self, messages, tools, response_format=None):

...

class UnionAlphaProvider(ModelProvider):

...

class FutureProvider(ModelProvider):

...

Union Alpha is configured externally:

MODEL_PROVIDER=union_alpha

MODEL_ID=stealth/union-alpha

No domain logic may depend on the model name.

16. Prompt architecture

System prompt sections:

1. role

2. domain rules

3. evidence taxonomy

4. tool policy

5. hypothesis policy

6. falsification requirement

7. uncertainty rules

8. provenance rules

9. output schema

Hard rule:

> Never invent a numerical measurement when a deterministic tool can calculate it.

17. Model context strategy

Because Union Alpha supports 262K context, the system may maintain a rich investigation context. However, the application should not dump every dataset row into the prompt.

Use:

• compact evidence summaries

• selected observations

• tool outputs

• references to cached raw data

• progressive context construction

This improves latency and reduces irrelevant context.

18. Multimodal strategy

The backend can generate:

• lap-time traces

• sector deltas

• tyre degradation plots

• position timelines

• strategy timelines

The agent may receive selected plots as images for visual cross-checking.

Images are secondary evidence. Numeric truth remains in deterministic outputs.

19. Evaluation architecture

Benchmark Question

■

▼

RACEWISE Agent

■

■■■ tool trace

■■■ evidence graph

■■■ final report

■

▼

Evaluation Harness

■

■■■ numerical correctness

■■■ evidence coverage

■■■ hypothesis quality

■■■ falsification

■■■ hallucination

■■■ reproducibility

Maintain a gold/reference answer for each benchmark question.

20. Security and privacy

• no secrets in model context

• environment variables for credentials

• tool sandboxing

• allow-listed tools

• read-only data tools

• counterfactual simulator isolated from external side effects

• prompt injection defenses for external text

• investigation logs sanitized before public demo

Union Alpha's current provider is anonymous; use only non-sensitive research data during the free experiment.

21. Repository structure

racewise/

■■■ apps/

■ ■■■ api/

■ ■■■ web/

■■■ packages/

■ ■■■ domain/

■ ■■■ data/

■ ■■■ analytics/

■ ■■■ investigation/

■ ■■■ agent/

■ ■■■ simulator/

■ ■■■ provenance/

■ ■■■ evaluation/

■■■ tests/

■ ■■■ unit/

■ ■■■ integration/

■ ■■■ benchmark/

■■■ datasets/

■ ■■■ fixtures/

■■■ docs/

■ ■■■ PRD.md

■ ■■■ ARCHITECTURE.md

■ ■■■ RESEARCH_BRIEF.md

■■■ scripts/

■■■ docker/

■■■ .env.example

■■■ README.md

22. Deployment

MVP target:

• React/Vite frontend

• FastAPI backend

• PostgreSQL

• containerized deployment

• SSE streaming

Optional:

• Vercel frontend

• managed Postgres

• container host for API

23. Architectural boundary with previous projects

RACEWISE may import existing capabilities only through adapters/interfaces.

It must not copy:

• Live F1 Intelligence's real-time publication layer

• RaceMind's complete ingestion architecture

• F1 Lap-Time Simulator's full physics engine

Instead:

• `F1DataAdapter` consumes data

• `SimulatorAdapter` exposes counterfactual capability

• `InvestigationEngine` is the new core IP

24. Future evolution

After MVP:

1. multi-model evaluation
2. richer causal models
3. probabilistic strategy simulation
4. multi-agent debate
5. cross-race knowledge base
6. automated research notebooks
7. uncertainty calibration
8. driver/team-specific engineering profiles

The one-week version must remain focused on the investigation loop.